Glossary and notation

This is a reference, not a chapter. Nothing here has to be read in order. When a symbol in one of the articles is unfamiliar, or you cannot remember which multiplication a particular dot means, look it up here and go back.

Two kinds of entry appear below. Sections 1 to 6 cover notation: the marks, operators and letters used in formulas. Sections 7 to 9 cover vocabulary: abbreviations, units, and where each idea is introduced in the course.

One warning about mathematical notation in general. It is not standardised. Different books use the same symbol for different things, and this glossary says what the symbol means *in this course*, with a note wherever a common alternative meaning exists elsewhere.

---

1. Standing conventions

Convention	Meaning
x, lowercase italic	a scalar or a vector, said in the text
A, uppercase italic	a matrix
$x^\top$	the transpose of x; vectors are columns, so $x^\top$ is a row
$n \times m$	a shape: n rows by m columns. The $\times$ here is the word "by", not an operation
x_i	entry i of x; A_{ij} is the entry in row i, column j
$i = 1 \ldots n$	maths counts from 1, Python counts from 0. The same index is `x[i-1]` in code
log	the natural logarithm, base e. A different base is written $\log_2$
$f(x)$ vs $f \circ g$	$f(x)$ applies f to x; $f \circ g$ is the function "do g, then f"

Shapes in the transformer modules use a standing set of letters:

Letter	Meaning	Typical value in the exercises
B	batch size, the number of sequences processed together	8 to 64
T	sequence length, the number of tokens	128 to 2,048
d or d_{model}	model width, the size of a token's vector	512 to 12,288
C	the same model width, in module 02 and in code following nanoGPT	512 to 12,288
h	number of attention heads. Careful: in module 04's activation formula, taken from Korthikanti et al., h is the model width and a is the head count	8 to 96
d_{head} or D	width of one head, usually d/h. Module 02 writes it $D = C/H$	64
V	vocabulary size	11 in the toy tasks, 50,000+ in real models
L	number of layers	4 in the exercises, 96 in GPT-3

Letter	Meaning	Typical value in the exercises
N	number of parameters, or number of data points. The text says which	

---

2. The kinds of multiplication

This is the notation that causes the most confusion, because at least five different operations are all called "multiplication" and several of them share a symbol.

Written	Name	Shapes	What it computes	In code
αA	scalar multiple	number, $n \times m$	every entry times α	`alpha * A`
AB, $A \cdot B$	matrix product	$(n \times k)(k \times m) \to n \times m$	entry (i, j) is row i of A dotted with column j of B	`A @ B`
Ax	matrix–vector	$(n \times m)(m) \to n$	the same rule, with one column	`A @ x`
$x^\top y$, $\langle x, y \rangle$, $x \cdot y$	dot, inner product	$(n)(n) \to$ scalar	$\sum_i x_i y_i$, one number	`x @ y`, `(x * y).sum()`
$xy^\top$	outer product	$(n)(m) \to n \times m$	entry (i, j) is $x_i y_j$; always rank 1	`torch.outer(x, y)`
$A \odot B$	Hadamard, elementwise	both $n \times m$	entry (i, j) is $A_{ij} B_{ij}$	`A * B`
$A \otimes B$	Kronecker product	$(n \times m)(p \times q) \to np \times mq$	every entry of A times the whole of B	`torch.kron(A, B)`
$\sum_i a_i b_i$	explicit sum	any	the general case; einsum is this with indices named	`torch.einsum(...)`

Three traps are worth knowing in advance:

- `*` in numpy and PyTorch is elementwise, never a matrix product. In MATLAB and in most textbooks, the same symbol means the matrix product. Use `@` for the matrix product in Python.
- Because of broadcasting, `A * B` with mismatched shapes often runs instead of raising an error, silently giving the wrong answer. Module 00 covers this in detail, and module 03 shows the bugs it causes.
- $\circ$ means function composition in this course. Some texts use it for the Hadamard product instead, which is why the articles use $\odot$ for elementwise multiplication throughout.

Reading an einsum

`torch.einsum("bhid,bhjd->bhij", q, k)` names the axes of each input, then names the axes of the output. Any letter that appears on the left but not on the right is summed over. Here `d` is missing from the output, so the operation multiplies and sums along `d`, which is the dot product of each query with each key. Everything else is carried along unchanged.

---

3. Marks on symbols

Mark	Example	Meaning here	Watch out
bar	$\bar{y}$, $\bar{O}$	the gradient of the loss with respect to that quantity, with the same shape as it. Also called the cotangent	in statistics a bar usually means an average, as in $\bar{x}$ for a sample mean
hat	$\hat{p}$	an estimate from data, or the empirical distribution of the data	also used for unit vectors, written $\hat{u}$, meaning u scaled to length 1
tilde	$\tilde{x}$	a modified or approximate version of x	
star	$x^\star$, $w^\star$	the optimal value, the one we are searching for	
superscript	$x^{(t)}$	the value at step or layer t, not a power	
transpose	$A^\top$	rows and columns swapped	
inverse	A^{-1}	the matrix undoing A; $A^{-1}A = I$	rarely computed in practice, even when written
absolute value	$\lvert x \rvert$	distance from zero, for one number	with a set, $\lvert S \rvert$ means how many elements it has
norm	$\lVert x \rVert$	the size of a whole vector or matrix; see section 4	
Kronecker delta	δ_{ij}	1 when $i = j$, otherwise 0	
indicator	$\mathbb{1}[\cdot]$	1 when the condition holds, otherwise 0	

---

4. Measuring size: norms

A norm turns a whole vector or matrix into one non-negative number that says how big it is. Which norm you choose changes what "big" means.

Norm	Formula	Also called	Where it appears
$\lVert w \rVert_1$	$\sum_i \lvert w_i \rvert$	L1, Manhattan norm	L1 penalty, lasso; drives weights to exactly zero (module 01)
$\lVert w \rVert_2$	$\sqrt{\sum_i w_i^2}$	L2, Euclidean norm	ordinary length; gradient clipping (module 01, 03)
$\lVert w \rVert_2^2$	$\sum_i w_i^2$	squared L2	L2 penalty, ridge regression, weight decay (module 01)
$\lVert w \rVert_\infty$	$\max_i \lvert w_i \rvert$	L-infinity, max norm	the size limit in adversarial attacks (module 08)
$\lVert A \rVert_F$	$\sqrt{\sum_{ij} A_{ij}^2}$	Frobenius norm	matrix error, as in low-rank and quantisation error (modules 01, 05)

The Frobenius norm is the L2 norm of a matrix flattened into one long vector. When the subscript on $\lVert \cdot \rVert$ is missing, L2 is meant.

---

5. Greek letters used in the course

Letter	Name	Standard meaning here
α	alpha	a generic coefficient or mixing weight; the strength of the load-balance loss (module 10)
β	beta	inverse temperature in softmax (module 01); the KL strength in RLHF and DPO (module 08); also Adam's β_1, β_2
γ	gamma	the scale parameter of a normalisation layer
Δ	capital delta	a change in something; the step size of a quantiser (module 05)
ϵ	epsilon	a tiny constant added for numerical safety; the size limit of an adversarial change (module 08)
η	eta	the learning rate
θ	theta	all the parameters of a model, taken together
κ	kappa	the condition number, $\lambda_{\max}/\lambda_{\min}$ (module 01)
λ	lambda	an eigenvalue; also a regularisation strength
μ	mu	a mean
ν	nu	a Lagrange multiplier (module 01)
π	pi	a policy, meaning the model being trained with RLHF; π_{ref} is the frozen model it started from, π^* the best policy (module 08)
ρ	rho	a correlation; the importance-sampling ratio $\pi_\theta/\pi_{\text{old}}$ in PPO (module 08)
σ	sigma	a singular value; a standard deviation; also the sigmoid function $\sigma(x)$
Σ	capital sigma	the diagonal matrix of singular values in the SVD. As an operator, a sum
τ	tau	the width of a prior (module 01); a temperature in some papers
ϕ	phi	a feature map or an auxiliary function
Ω	capital omega	a regularisation term, in some references

---

6. Other symbols

Symbol	Read as	Meaning
$\sum_{i=1}^{n}$	sum over i	add up the terms for $i = 1$ to n
$\prod$	product over	multiply the terms together
$\mathbb{E}[x]$	expectation of x	the average of x over a distribution
$p(a \mid b)$	probability of a given b	the conditional probability; the bar is "given", not division
$x \sim p$	x is drawn from p	x is sampled from the distribution p
$\propto$	proportional to	equal up to a constant factor, which often cancels later
$\approx$	approximately equal	
$\ll, \gg$	much less than, much greater than	
$\in$	in	$x \in S$ means x is an element of the set S
∇f	gradient of f	the vector of all partial derivatives of f

Symbol	Read as	Meaning
$\partial L/\partial x$	partial derivative	how L changes when x changes and everything else is held fixed
$\arg\min_x f(x)$	argmin	the x that makes $f(x)$ smallest, not the smallest value itself
$\mathcal{O}(n^2)$	big-O of n^2	the cost grows no faster than n^2, ignoring constants
$D_{\mathrm{KL}}(p \,\|\, q)$	KL divergence from q to p	how far q is from p; never negative, and not symmetric
$H(p)$	entropy of p	the average surprise of samples from p
$\mathrm{diag}(a, b)$	diagonal matrix	the matrix with a and b on its diagonal and zeros elsewhere
I	the identity matrix	ones on the diagonal, zeros elsewhere

7. Abbreviations

Maths and training

Short	Full	One line
SVD	singular value decomposition	writes any matrix as rotate, stretch, rotate (module 01)
PCA	principal component analysis	the SVD applied to centred data
VJP / JVP	vector–Jacobian / Jacobian–vector product	one step of reverse / forward mode autodiff (module 00)
MLE	maximum-likelihood estimation	pick the parameters that make the data most likely (module 01)
MAP	maximum a posteriori	MLE with a prior, which is where regularisation comes from (module 01)
NLL	negative log-likelihood	the average surprise of the data under the model (module 01)
KL	Kullback–Leibler divergence	distance from one distribution to another (modules 01, 08)
CE	cross-entropy	the usual classification loss; equals NLL for one-hot targets
SGD	stochastic gradient descent	gradient descent on small batches
LN	LayerNorm	normalises each token's vector (modules 02, 03)
RMSNorm	root-mean-square norm	LayerNorm without the mean subtraction (module 02)
GELU / SwiGLU	activation functions	smooth alternatives to ReLU (module 02)
RoPE	rotary position embedding	encodes position by rotating query and key pairs (module 02)
MHA	multi-head attention	attention run in several parallel subspaces (modules 00, 02)
LoRA	low-rank adaptation	fine-tunes with a low-rank update instead of full weights (module 01)

Systems and training at scale (module 04)

Short	Full	One line
DP / TP / PP	data / tensor / pipeline parallelism	the three ways to split training across GPUs
all-reduce	—	every GPU ends up with the sum of what all of them had
reduce-scatter	—	every GPU ends up with its own slice of that sum
all-gather	—	every GPU ends up with all the slices
all-to-all	—	every GPU gets the slice each other GPU addressed to it
NCCL	NVIDIA collective communications library	the library that implements those four on GPUs
ZeRO	zero redundancy optimiser	splits optimiser state, gradients and weights across GPUs
FSDP	fully sharded data parallel	PyTorch's implementation of the last ZeRO stage
1F1B	one forward, one backward	a pipeline schedule that starts backward passes early
MoE	mixture of experts	a layer that stores E feed-forward networks and runs k of them per token (module 10)
EP	expert parallelism	putting different experts on different GPUs; needs two all-to-alls per layer
router	—	the linear layer that scores every expert for a token, and whose top k are used
gate	—	the router probability of a chosen expert, used as the weight on its output
capacity factor	—	how many token slots each expert is given, as a multiple of the even share
token dropping	—	skipping a token whose chosen expert is already full; it passes through the residual
MFU	model FLOPs utilisation	the fraction of a GPU's peak arithmetic you actually use
ψ	—	one copy of the model's weights, used to size ZeRO's traffic (module 04)
$2(n-1)/n$	—	the copies each GPU sends in a ring all-reduce; rises towards 2 and stops
AMP	automatic mixed precision	compute in 16-bit, keep a 32-bit copy of the weights
HBM	high-bandwidth memory	the GPU's main memory (modules 05, 07, 09)
SRAM	static RAM	the small, fast memory on the GPU chip (modules 07, 09)

Inference (module 05)

Short	Full	One line
KV cache	key–value cache	stores past keys and values so each new token is cheap
MQA / GQA	multi-query / grouped-query attention	share keys and values across heads to shrink that cache
PTQ / QAT	post-training quantisation / quantisation-aware training	quantise after training, or train with it in the loop
GPTQ / AWQ	two weight-quantisation methods	both protect the weights that matter most
TTFT / TPOT	time to first token / time per output token	the two latencies users feel

Retrieval (module 06)

Short	Full	One line
RAG	retrieval-augmented generation	retrieve passages, then answer from them
BM25	best match 25	the standard keyword ranking function
IDF	inverse document frequency	the weight that makes rare words count more
MRR	mean reciprocal rank	1 divided by the rank of the first correct result, averaged
nDCG	normalised discounted cumulative gain	a ranking score that rewards putting good results first
RRF	reciprocal rank fusion	combines two rankings without tuning weights
IVF / HNSW	inverted file / hierarchical navigable small world	two approximate nearest-neighbour indexes

Safety (module 08)

Short	Full	One line
RLHF	reinforcement learning from human feedback	train a reward model from comparisons, then optimise it
REINFORCE	(not an abbreviation)	the policy gradient: reward-weighted cross-entropy on the model's own samples
PPO	proximal policy optimisation	REINFORCE plus a baseline, a ratio and a clip that bounds how far one batch can move the policy
GRPO	group relative policy optimisation	PPO with the mean of a group of answers as the baseline, so no value model is needed
DPO	direct preference optimisation	reach the same optimum without a separate reward model
CAI	constitutional AI	use a written set of principles instead of human labels

Short	Full	One line
FGSM / PGD	fast gradient sign method / projected gradient descent	one-step and many-step adversarial attacks
DP-SGD	differentially private SGD	clip per-example gradients and add noise
ROC	receiver operating characteristic	TPR plotted against FPR as the threshold sweeps
AUC	area under the ROC curve	the chance a positive scores above a negative
TPR	true positive rate, also recall	$TP/(TP+FN)$, the share of positives caught
FPR	false positive rate	$FP/(FP+TN)$, the share of negatives wrongly caught
AP	average precision	area under the precision–recall curve; use it when positives are rare

GPU (module 09)

Short	Full	One line
SM	streaming multiprocessor	one of the GPU's independent processors
GEMM / SGEMM	general matrix multiply, single-precision	the standard matrix-multiplication routine
cuBLAS	NVIDIA's BLAS library	the tuned implementation you are measured against

---

8. Units and quantities

Unit	Meaning	Trap
FLOP	one floating-point operation	a count
FLOP/s	floating-point operations per second	a rate. "FLOPs" in papers can mean either; the context decides
GB vs GiB	10^9 bytes vs 2^{30} bytes	GiB is 7.4% larger. Hardware is sold in GB, allocators report GiB
bytes per parameter	memory per weight	2 for bf16 weights, 16 for a parameter under Adam in mixed precision (module 04)
nat vs bit	units of entropy and loss	natural log gives nats, $\log_2$ gives bits. 1 nat = 1.443 bits
perplexity	e^{NLL}	the effective number of equally likely choices
token	the unit a model reads and writes	roughly 0.75 English words
tok/s	tokens per second	the usual throughput measure for generation

Number formats, which decide most of the memory arithmetic:

Format	Bits	Exponent / mantissa bits	Note
fp32	32	8 / 23	the default in PyTorch
fp16	16	5 / 10	more precision than bf16, far smaller range
bf16	16	8 / 7	same range as fp32, which is why training prefers it
int8	8	integer	with a scale factor per tensor, channel or group (module 05)
int4	4	integer	weight-only in practice

---

9. The same quantity under different letters

The course follows each field's own conventions rather than imposing one notation, because that is what the papers and the code you will read do. The cost is that one quantity changes letter between modules. This table is the map.

Quantity	Letters used	Where
model width	d, d_{model}	modules 00, 01, 05, and most papers
	C	module 02, following nanoGPT's (B, T, C)
	h	module 04's activation formula, following Korthikanti et al.
sequence length	T	modules 00, 01, 02, 07
	s	modules 04, 05
batch size	B	modules 00, 02
	b (micro-batch)	modules 04, 05
number of heads	H	module 02
	a	module 04
	n_{kv} (key–value heads)	module 05
width of one head	D	module 02
	d_{head}, d_k	modules 01, 05, 06
number of layers	L	modules 02, 04, 05
the loss	$\mathcal{L}$	modules 01, 08

A few letters also carry different meanings in different modules. Each is defined where it is used, but it is worth knowing in advance:

Letter	Meanings	Where
V	the SVD's right factor; attention's values; vocabulary size	01; 00, 01, 02, 07; 03
C	the product AB in the matmul rule; model width	01, 09; 02
N	parameters; observations; passages in a corpus	04, 05; 01; 06
n	GPUs; examples; matrix size; passages containing a term	04; 02; 09; 06

Letter	Meanings	Where
m	a matrix dimension; a token position; micro-batches; a running maximum	01; 02; 04; 07
k	an inner dimension; cluster count; BM25's k_1; RRF's constant; accumulation steps	01; 02; 06; 06; 04
p	probabilities; pipeline stages; convolution padding	01, 05, 08; 04; 02
q	a quantised integer; the draft model's distribution; a query	05; 05; 06
β	inverse temperature; the KL strength; Adam's averaging rates	01; 08; 04
γ	a normalisation layer's scale; drafted tokens per round	02; 05
σ	a singular value; a standard deviation; the sigmoid function	01; 02; 01, 08
ϵ	a finite-difference step; a guard against dividing by zero; an attack budget	02; 02; 08

10. Where each idea is introduced

Idea	Module	Section
tensors, strides, broadcasting	00	1, 2
autograd as vector–Jacobian products	00	3
numerically stable softmax	00	4
SVD, rank, low-rank approximation	01	1, 2
condition number and convergence speed	01	3
the gradient of attention, by hand	01	5
cross-entropy as maximum likelihood	01	7
softmax from maximum entropy	01	8
L2 regularisation as a Gaussian prior	01	9
writing a transformer and its backward pass	02	all
finding bugs that do not raise errors	03	all
memory, parallelism and scaling laws	04	all
KV cache, quantisation, speculative decoding	05	all
chunking, BM25, embeddings, rerankers	06	all
reading a paper, and checking its claims	07	all
RLHF, DPO, reward hacking, evaluation	08	all
GPU memory hierarchy and matrix-multiply kernels	09	all
routing, load balance, expert parallelism	10	all

Where to look next

- Kevin Murphy, *Probabilistic Machine Learning: An Introduction* (2022). Its notation chapter is the closest thing to a standard for this material.

- Gilbert Strang, *Introduction to Linear Algebra*. The source for sections 1 to 4 above.
- *The Matrix Cookbook*, Petersen and Pedersen. A dense reference of matrix identities, including every derivative rule used in module 01.
- Terence Parr and Jeremy Howard, *The Matrix Calculus You Need For Deep Learning* (https://explained.ai/matrix-calculus/). Careful on exactly the notation that this glossary compresses into a table.